Vision à présenter demain : 2 niveaux d'évolution du chatbot ST

Je te propose une présentation en **2 approches progressives** :

- **Niveau 1** : avec la base actuelle (**issues + files**) et l'outil ST actuel
  → transformer le chatbot en **assistant de diagnostic de nouvelles issues**
- **Niveau 2** : avec une base enrichie (**solution-oriented**)
  → aller vers un **assistant de résolution plus avancé**

Comme ça, tu montres :

- ce qui est **faisable maintenant**,
- ce qui est **faisable ensuite**,
- et les **résultats attendus** à chaque étape.

1. Niveau 1 — Assistant de diagnostic pour nouvelles issues

1.1. Objectif

Passer de :

- "retrouver une issue similaire" à
- "analyser une nouvelle issue à partir des cas connus et de la documentation"

Le système ne donne pas forcément un fix confirmé, mais il produit :

1. **classification du problème**
2. **faits proches trouvés dans la KB**
3. **hypothèses plausibles**
4. **actions de debug recommandées**
5. **workarounds éventuels**
6. **inconnus / limites**

1.2. Base utilisée

Sans changer drastiquement le pipeline :

- `st_ready_issues_stm32cubeh7.json`
- `st_ready_files_stm32cubeh7.json`

Donc on reste dans l'existant :

- issues enrichies et reformattées
- fichiers documentaires préparés pour ST

1.3. Mécanisme technique

Étape 1 — Ingestion dans ST KB

Tu l'as déjà :

- upload JSON issues
- upload JSON files
- chunking interne ST :
  - `PARENT_CHILD`
  - `childChunkSize=1000`
  - `parentChunkSize=3000`

Étape 2 — Récupération multi-source

Quand l'utilisateur soumet une **nouvelle issue**, le moteur :

- récupère des passages pertinents depuis :
  - les issues ST-ready,
  - les files ST-ready.

On ne cherche plus seulement une "meilleure issue similaire", mais plusieurs **éléments de contexte**.

Étape 3 — Prompt de diagnostic structuré

C'est ici que tu changes de paradigme.

Prompt type :

```text
Réponds uniquement à partir de la base STM32CubeH7 (issues + files).
N'invente aucun correctif confirmé absent de la base.

Pour cette nouvelle issue :
1. Identifie le composant, layer, board, type probable de problème.
2. Résume les faits confirmés retrouvés dans la base.
3. Formule des hypothèses plausibles, clairement marquées comme non confirmées.
4. Propose les actions de debug prioritaires.
5. Mentionne les workarounds documentés s'ils existent.
6. Indique ce qui reste inconnu.

Objectif : aider une équipe de maintenance à investiguer plus vite.
```

Étape 4 — Sortie cadrée

Format attendu :

```text
## Problem classification
## Confirmed evidence from KB
## Plausible hypotheses
## Recommended debug actions
## Potential workarounds
## Unknowns / validation needed
```

1.4. Ce qu'il faut techniquement pour l'implémenter

Côté données

Tu as déjà l'essentiel :

- issues ST-ready
- files ST-ready

Côté configuration / usage

Il faut surtout :

- définir les **prompts de diagnostic**,
- préparer des **cas de test "new issues"**,
- évaluer si les réponses deviennent plus utiles qu'un simple résumé.

Côté évaluation

Créer un benchmark dédié :

- questions "nouvelle issue"
- critères :
  - bonne classification du problème
  - pertinence des hypothèses
  - qualité des actions proposées
  - faible hallucination
  - ancrage dans la KB

1.5. Faisabilité

Très faisable à court terme

Parce que :

- tu n'as pas besoin de changer l'outil ST,
- tu n'as pas besoin de nouveaux pipelines massifs,
- tu réutilises :
  - ta KB actuelle,
  - ta méthodologie d'évaluation,
  - mais avec une nouvelle forme d'interrogation.

Effort estimé

Faible à modéré :

- design prompts
- définir benchmark
- tester et analyser

1.6. Résultats attendus

Résultats réalistes

Le chatbot devrait être capable de :

- reconnaître le composant / contexte du nouveau bug,
- relier le nouveau bug à plusieurs cas connus,
- produire des **hypothèses plausibles**,
- proposer un **plan de debug utile**,
- éviter le simple "I DON'T KNOW".

Limites

Il ne pourra pas encore :

- garantir un fix exact,
- générer un patch confirmé,
- remplacer l'équipe maintenance.

👉 **Valeur attendue** :

> gain de temps important dans la phase de triage et de diagnostic initial.

2. Niveau 2 — Assistant de résolution avancé

Là, on va plus loin que le diagnostic.

2.1. Objectif

Passer de :

- "voici ce qu'on sait et quoi tester" à
- "voici les directions de résolution les plus plausibles, basées sur des correctifs historiques et des patterns de code"

Le système commence à approcher une aide à la correction.

2.2. Nouvelle brique nécessaire : Solution-Oriented Knowledge Layer

Ajouter une nouvelle base, par exemple :

- commits
- PRs
- release note fixes
- workarounds
- éventuellement code ciblé / diffs

Cette base formerait une **Solution DB**.

2.3. Mécanisme technique

Étape 1 — Construire une Solution DB

Objets indexés :

- commit docs
- PR docs
- fix summaries
- workaround cards

Exemple de document :

```json
{
  "id": "fix-usb-abc123",
  "source_kind": "solution",
  "repo": "STM32CubeH7",
  "component": "USB",
  "layer": "Middleware",
  "problem_summary": "Composite CDC/HID could hang under concurrent transfers",
  "root_cause_summary": "Shared endpoint state not reset correctly",
  "solution_summary": "Separated class state handling and fixed transfer completion path",
  "files_changed": ["usbd_cdc.c", "usbd_customhid.c"],
  "clean_text": "Problem: ... Root cause: ... Solution: ... Validation: ..."
}
```

Étape 2 — Double retrieval

Lors d'une nouvelle issue :

- Retrieval 1 : **Problem KB**
  - pour comprendre le bug
- Retrieval 2 : **Solution DB**
  - pour récupérer des patterns de correction proches

Étape 3 — Réponse solver

Le modèle produit :

```text
## Confirmed facts
## Confirmed fixes from similar cases
## Plausible hypotheses
## Candidate patch directions
## Recommended validation
## Unknowns
```

2.4. Extension possible : analyse de code

Encore plus avancé :

- indexer certains fichiers code du repo,
- ou faire un `code_search_tool`
- ou analyser des diffs / fonctions concernées

Exemple :

- si bug USB composite,
- regarder automatiquement :
  - `usbd_cdc.c`
  - `usbd_customhid.c`
  - `usbd_composite.c`

L'assistant pourrait alors dire :

- quelles zones du code semblent concernées,
- quels états / callbacks / buffers sont suspects.

2.5. Ce qu'il faut techniquement

Nouvelles données

- commits
- PRs
- release notes orientées fixes
- éventuellement code

Nouveau pipeline

Nouveaux modules :

- ingestion commits/PRs
- cleaning
- conversion en docs solution
- chunking
- indexation

Nouvelle logique de réponse

- double base (problem + solution)
- prompt solver plus élaboré
- éventuellement agent multi-tools plus tard

2.6. Faisabilité

Faisable, mais plus coûteux

Comparé au niveau 1 :

- plus de développement
- plus de data engineering
- plus de validation
- plus d'évaluation nécessaire

Faisabilité globale

Oui, mais :

- pas "pour demain",
- plutôt comme **extension ambitieuse et crédible** du PFE

2.7. Résultats attendus

Résultats réalistes

Le système pourrait :

- proposer des **directions de correction**,
- retrouver des **fixes proches**,
- suggérer les **fichiers / couches à inspecter**,
- être bien meilleur sur les bugs nouveaux que le RAG simple.

Limites

Même là :

- pas de garantie de correctif parfait,
- nécessité de validation humaine / tests hardware,
- surtout sur embarqué.

👉 **Valeur attendue** :

> passer d'un support documentaire à un véritable assistant de résolution technique.

3. Comment présenter les 2 niveaux dans ta slide

Tu peux faire une slide simple comme ça :

Évolution du chatbot STM32Cube : 2 niveaux

Niveau 1 — Diagnostic assistant (court terme, faisable)

Base : issues + files ST-ready
But :

- analyser une nouvelle issue,
- retrouver plusieurs cas/documents proches,
- produire :
  - classification,
  - hypothèses plausibles,
  - actions de debug,
  - workarounds éventuels.
    Faisabilité : élevée
    Valeur : améliore fortement le triage et le diagnostic initial.

Niveau 2 — Resolution assistant (moyen terme, plus ambitieux)

Base : issues + files + Solution DB (commits, PRs, fixes, workarounds)
But :

- relier un nouveau bug à des patterns de résolution,
- proposer des directions de correction,
- pointer des zones de code / middleware concernées.
  Faisabilité : moyenne à élevée, mais demande de nouvelles briques
  Valeur : assistant technique avancé orienté résolution.

4. Message clé à dire demain

Tu peux dire ceci :

> "Nous avons identifié deux niveaux d'ambition.
> Le premier, immédiatement faisable avec la base actuelle et l'outil ST, consiste à faire évoluer le chatbot vers un **assistant de diagnostic de nouvelles issues** : il ne se limite plus à retrouver des issues similaires, mais structure les faits, formule des hypothèses plausibles et propose des actions de debug.
>
> Le second niveau, plus ambitieux, consisterait à ajouter une **couche orientée solutions** à partir des commits, PRs et correctifs historiques, afin d'aller vers un **assistant de résolution avancé** capable de suggérer des directions de correction sur des bugs nouveaux."

5. Résumé ultra-court pour ta présentation

Niveau 1

- **Approche** : RAG ST + prompts de diagnostic
- **Input** : issues + files
- **Output** : hypothèses + debug plan
- **Faisabilité** : forte
- **Résultat attendu** : meilleure gestion des nouvelles issues

Niveau 2

- **Approche** : RAG ST + Solution DB + éventuellement code analysis
- **Input** : issues + files + fixes historiques
- **Output** : directions de correction / patterns de fix
- **Faisabilité** : plus lourde
- **Résultat attendu** : assistant de résolution plus avancé

Si tu veux, je peux maintenant te préparer :

1. un **schéma Mermaid** pour cette slide,
2. ou un **discours oral de 1 minute** pour expliquer ces deux niveaux demain.